LLD Interview Practice - Coding Instructions

> These are the rules and style guide for ALL LLD files in this project.
> Every file should be completable in a **60-minute interview** setting.

---

1. GOAL: Interview-Ready, Not Production-Ready

- Write code you can **type in 45-50 minutes** leaving 10-15 min for discussion.
- Prioritize **working logic** over boilerplate.
- Every file must compile and run with `javac File.java && java File`.
- Single file only — all classes in one `.java` file.

---

2. NO BOILERPLATE — Focus on Logic

❌ DO NOT write:

- Verbose getters/setters (use `public final` fields or one-liner getters only if needed)
- `toString()`, `equals()`, `hashCode()` unless they ARE the logic
- Javadoc on obvious methods
- Separate files for enums, exceptions, or small classes

✅ DO write:

- **Algorithms and data structures** — the meat of the problem
- **Design patterns** where they naturally fit (Strategy, Observer, Builder)
- **Custom exceptions** with minimal fields (just message + 1-2 context fields)
- **Interfaces** for extensibility (different strategies, pluggable components)
- Inline field access: `doc.id` or `public final String id` over `getId()`

2.1. NO DEAD CODE — Every Field Must Earn Its Place

Before adding any field, enum value, or method, ask: **"Is this read/used anywhere?"**

❌ REMOVE if:

- **Write-only fields** — incremented/set but never read (e.g., `accessCount++` but never displayed or used in logic)
- **Unused enum values** — `CANCELLED` in MatchStatus if no code ever sets or checks it
- **Decorative fields** — `name` when `id` is sufficient and `name` is never displayed
- **Dead methods** — `getTier()` that nothing calls, `toString()` that's never printed
- **Phantom metrics** — `bytesServed`, `wins`, `matchesPlayed` that are tracked but never queried
- **Unused timestamps** — `createdAt`, `startedAt` if no logic depends on them

✅ KEEP if:

- Field is **read in logic** (e.g., `usedBytes` checked against `maxBytes` for eviction)
- Field is **displayed in output** (e.g., `skillRating` shown in status display)
- Field is **used in algorithm** (e.g., `queuedAt` for wait-time-based widening)
- Enum value is **checked in a switch/if** somewhere

Rule of thumb:

> If you `grep` for a field and only find its declaration + assignment (never read), delete it.

```
// ❌ BAD — write-only counter nobody reads
int accessCount;
void touch() { accessCount++; }  // incremented but NEVER read

// ❌ BAD — name when id is enough
class Player {
    final String playerId;
    final String name;        // never displayed, id used everywhere
    final int skillRating;
}

// ✅ GOOD — lean, every field serves a purpose
class Player {
    final String playerId;
    final int skillRating;    // used in matching algorithm
    PlayerStatus status;      // checked for IDLE/QUEUED/IN_MATCH
    LocalDateTime queuedAt;   // used for wait-time widening
}
```

2.2. TODO / HINT FORMAT (Practice Mode)

Files should provide **structure + hints + TODOs** — the user solves them.

What to provide:

- Full class/interface/enum definitions with fields
- Method signatures with clear `// TODO: Implement` markers
- `// HINT:` comments showing approach (not full code)
- Working test cases in `main()` that validate the solution
- Exceptions, enums, domain classes fully written (not the interesting part)

What NOT to provide:

- **Complete method implementations** — leave as TODO for user to solve
- Full solution code in hint comments — just the approach/data structure hint

Example:

```java
// ✅ GOOD — hints have actual code lines (commented out), user
uncomments/types
TollTransaction processToll(String plate, String boothId, TripType
tripType)
        throws InsufficientBalanceException, InvalidPassException {
    // TODO: Implement
    // HINT: Vehicle v = vehicles.get(plate);
    // HINT: if (v == null) throw new IllegalArgumentException("Unknown
vehicle: " + plate);
    // HINT: double toll = pricing.calculate(v.type, tripType);
    // HINT: if (v.hasPass()) {
    // HINT:     TollPass pass = passes.get(v.passId);
    // HINT:     if (pass == null || !pass.active) throw new
InvalidPassException(v.passId);
    // HINT:     pass.deduct(toll);
    // HINT:     method = PaymentMethod.TOLL_PASS;
    // HINT: } else {
    // HINT:     method = PaymentMethod.CASH;
    // HINT: }
    // HINT: TollTransaction txn = new TollTransaction(plate, boothId,
toll, method, tripType);
    // HINT: transactions.add(txn);
    // HINT: return txn;
    return null;
}

// ❌ BAD — vague hints with no code
// HINT: 1. Lookup vehicle, validate exists
// HINT: 2. Calculate toll
// HINT: 3. Process payment
```

Style example:

```java
// ✅ GOOD — compact, logic-focused
class Document {
    final String id, title, content;
    final long createdAt = System.currentTimeMillis();

    Document(String id, String title, String content) {
        this.id = id; this.title = title; this.content = content;
    }
}

// ❌ BAD — bloated, wastes interview time
class Document {
    private String id;
    private String title;
    private String content;
```

```
    public Document(String id, String title, String content) { ... }
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    @Override public String toString() { return "Document{id=...}"; }
    @Override public boolean equals(Object o) { ... }
    @Override public int hashCode() { ... }
}
```

---

3. CONCURRENCY IS EXPECTED

Make systems **thread-safe by default** unless the problem is purely algorithmic.

Use these patterns:

What	When
`ConcurrentHashMap`	Default choice for shared maps
`ReadWriteLock`	Multiple readers, exclusive writer (search index, cache)
`synchronized` on method	Simple mutual exclusion (rate limiter, booking)
`AtomicInteger/Long`	Counters, IDs
`ExecutorService`	In tests to prove thread safety
`volatile`	Flags read by multiple threads

Always include a **thread-safety test** in `main()`:

```
// Thread safety test
ExecutorService exec = Executors.newFixedThreadPool(4);
List<Future<?>> futures = new ArrayList<>();
for (int i = 0; i < 100; i++) {
    final int idx = i;
    futures.add(exec.submit(() -> system.doOperation(idx)));
}
for (Future<?> f : futures) f.get(); // wait for all
exec.shutdown();
System.out.println("✓ Thread-safe: " + system.getCount() + " operations
completed");
```

---

4. FILE STRUCTURE (Top to Bottom)

```
1. imports
2. Top comment block: Problem statement, Requirements, Key data
structures, Complexity
3. Custom Exceptions (if needed, keep minimal)
4. Enums (inline, compact)
5. Interfaces / Strategy definitions
6. Core data structure classes (the algorithm)
7. Service/Engine class (main orchestrator)
8. public class MainFile { main() with tests }
9. Bottom comment: Interview discussion notes (concise)
```

---

5. TOP COMMENT BLOCK (Required)

Every file starts with a concise problem summary:

```
/*
 * SYSTEM_NAME - Low Level Design
 * ================================
 *
 * REQUIREMENTS:
 * 1. ... (functional requirements, 5-8 bullets)
 *
 * KEY DATA STRUCTURES:
 * - What -> Why (e.g., "Inverted Index: term -> docId for O(1) lookup")
 *
 * DESIGN PATTERNS:
 * - Strategy/Observer/etc. if used
 *
 * COMPLEXITY:
 *   operation1: O(...)
 *   operation2: O(...)
 */
```

---

6. TEST CASES IN `main()`

- **10-12 test cases** covering:
    1. Happy path / basic functionality
    2. Multiple operations
    3. Edge cases (null, empty, not found)
    4. Algorithm correctness (ranking, ordering, limits)
    5. Concurrent operations (thread safety)
    6. Scale test (100-1000 items, measure time)
- Use **clear section headers**:

```
System.out.println("=== Test 3: Edge Cases ===");
```

- Use ✓ **checkmarks** for assertions:

```
System.out.println("✓ Returned 0 results for empty query");
```

---

7. INTERVIEW DISCUSSION NOTES (Bottom)

Keep concise — **30-50 lines max**. Cover:

1. **Core algorithm/data structure** — why this choice, alternatives
2. **Time & space complexity** — table format
3. **Scalability** — sharding, replication, caching (2-3 lines)
4. **Trade-offs** — what you'd do differently at scale
5. **Real-world systems** — name 1-2 (Elasticsearch, Redis, Kafka, etc.)

```
/*
 * INTERVIEW NOTES:
 * 1. CORE: Inverted index for O(1) term lookup...
 * 2. COMPLEXITY: search O(q*d), index O(n)...
 * 3. SCALE: Shard by docId, replicate for reads...
 * 4. REAL-WORLD: Elasticsearch = Lucene + sharding
 */
```

---

8. DESIGN PATTERNS — Use When Natural

Pattern	When to Use	Example
Strategy	Multiple algorithms for same task	Rate limiter (TokenBucket vs FixedWindow)
Observer	Event notifications	PubSub, webhooks, notifications
Builder	Complex object construction	Query builder, config builder
Factory	Create variants of a type	Limiter factory, parser factory
Command	Encapsulate operations	Undo/redo, job scheduler
State	Object behavior changes by state	Vending machine, order status
Decorator	Add behavior dynamically	Logging, caching, rate limiting layers

---

9. COMMON IMPORTS

```java
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.locks.*;
import java.util.concurrent.atomic.*;
import java.util.stream.*;
import java.util.function.*;
```

---

10. WHAT MAKES A GREAT LLD IN INTERVIEW

Must Have (60%+ of score):

- **Correct core algorithm** — the data structure/algorithm actually works
- **Clean class design** — right separation of concerns
- **Working code** — compiles and runs, handles basic cases

Differentiators (remaining 40%):

- **Thread safety** — shows systems thinking
- **Edge case handling** — null checks, bounds, empty inputs
- **Extensibility** — interfaces, strategy pattern, easy to add new types
- **Complexity awareness** — can explain Big-O of each operation
- **Scalability discussion** — knows how to shard, cache, replicate

Time Budget (60 min):

```
0-5 min:   Clarify requirements, discuss approach
5-10 min:  Define classes, interfaces, data structures
10-45 min: Implement core logic + tests
45-55 min: Run, fix bugs, add edge cases
55-60 min: Discuss scalability, trade-offs
```

---

11. QUICK REFERENCE — Data Structure Choices

Problem	Data Structure	Why
Fast lookup by key	`HashMap` / `ConcurrentHashMap`	O(1) get/put
Ordered by score/priority	`TreeMap` / `PriorityQueue`	O(log n) insert/remove
LRU eviction	`LinkedHashMap` with access order	O(1) get + evict
Prefix matching	`Trie`	O(k) where k = prefix length
Range queries	`TreeMap`	O(log n) floor/ceiling
Graph traversal	`Map<Node, List<Edge>>`	Adjacency list

Problem	Data Structure	Why
Counting/frequency	`Map<Key, AtomicInteger>`	Thread-safe counting
Time-series data	`Deque<TimestampedEntry>`	Sliding window
Unique items + order	`LinkedHashSet`	Insertion order + O(1) contains

12. NAMING CONVENTIONS

- **Engine/Service** suffix for main orchestrator: `SearchIndexEngine`, `RateLimiterService`
- **System** suffix for the public main class: `SearchIndex`, `RateLimiterSystem`
- **Strategy interfaces**: verb-based like `RateLimiter`, `PricingStrategy`
- **Enums**: `ALL_CAPS` values: `RequestResult.ALLOWED`, `SeatStatus.BOOKED`
- **Exceptions**: descriptive `XNotFoundException`, `XExceededException`

Last updated: March 2026